

Feasibility of Homomorphic Inference for a Genomic Foundation Model: A Client-Assisted CKKS Evaluation of DNAGPT

Christos Galanopoulos¹, Kimon Antonios Provatas¹, Ilias Georgakopoulos-Soares^{1,*}

¹The University of Texas at Austin, Austin, TX, USA

*Correspondence: Ilias Georgakopoulos-Soares

Homomorphic encryption · Genomic privacy · Transformer inference · GPU acceleration

Abstract

Motivation. Genomic sequence identifies its contributor and cannot be replaced once it has been disclosed. When a genomic model is available only as a service, using it ordinarily means handing that sequence to the provider. We ask whether a provider can instead run a released genomic transformer for a data owner who never reveals the input in plaintext, and where numerical agreement, memory, or cost would stop such a deployment.

Results. We first reproduce DNAGPT on three genomic task families and freeze its per-example plaintext predictions; the encrypted run must then reproduce them. The work is then split under a single key. The provider evaluates the model’s encrypted arithmetic on ciphertexts. The data owner, who holds the secret key in any case, decrypts at points fixed before execution, evaluates exactly the three functions CKKS cannot compute directly, the LayerNorm inverse square root, causal softmax, and GELU, and returns freshly encrypted values. All 12 released blocks and the task head execute this way from encrypted embedded vectors, with the released weights and nonlinear formulas unmodified and with no homomorphic bootstrap at any point. For the one prompt evaluated, the decrypted label matches the reference. Its margin has 8.56×10^{-9} relative error, 6.67 orders of magnitude below the 4×10^{-2} tolerance fixed in advance, and no polynomial approximation of a nonlinearity contributes to that residual. Each return to the data owner restarts the computation from a fresh ciphertext, so the multiplicative depth the circuit must provide is set by the longest stretch between two such points rather than by the depth of the whole network. It stays at 13 for the complete model, and device-wide GPU memory used peaks at 9839 MiB without growing from one block to the next. One in-process execution takes 1.86 hours on an A100 GPU node. A pure non-interactive configuration, which avoids the interaction altogether, completes one released-weight block but then exceeds the tested 80 GB accelerator-memory envelope while loading the cryptographic material needed to compose the next. That barrier therefore lies in the polynomial treatment of the nonlinearities rather than in the model’s dense maps.

Conclusions. The executed case establishes complete-model graph feasibility and numerical agreement for one input, and locates the limiting barrier in throughput rather than in correctness or accelerator memory. Return-

ing to the key holder is what makes the twelve-block encrypted computation expressible without bootstrapping, because it caps the depth any single encrypted stretch has to carry; bounding the cached plaintexts per stage separately keeps host memory from accumulating. The current single-process implementation is far too slow for interactive use and has not been evaluated as a networked service, so feasibility and practicality are reported as separate verdicts.

Availability. Code, evaluation harnesses, and the evidence record are maintained at https://github.com/Georgakopoulos-Soares-lab/private_genomic_dnagpt.

Key Points

- **The complete released model runs on encrypted input.** All twelve DNAGPT blocks and the task head run at the task’s own 103-token prompt, starting from encrypted input vectors, under one key and with no bootstrapping step. Nothing in the released model is changed to make this possible. For the one prompt evaluated, the encrypted run returns the same label as the plaintext model, with 8.56×10^{-9} relative error on the classification margin, 6.67 orders of magnitude below the 4×10^{-2} tolerance set in advance.
- **The encrypted result is checked against a reference that is itself checked.** Two independently written plaintext implementations of the released model must agree in double precision before either is used, and only then is the encrypted output compared with them. Agreement therefore reflects the model’s own computation rather than one implementation reproducing its own error.
- **Handing the nonlinear steps back to the key holder keeps the encrypted circuit shallow.** Each return restarts the computation from a fresh ciphertext, so the circuit only has to be deep enough for the longest stretch between two returns rather than for the whole network. That depth stays at 13 for the complete model, and device-wide GPU memory used peaks at 9839 MiB with no growth from one block to the next. A separate limit on how many encoded weights stay in memory at once keeps host memory bounded, with the checked operation schedule unchanged.

- **Feasible, not yet practical.** One in-process execution takes 1.86 hours and needs the key holder online throughout, which is far too slow for interactive use. The protocol is interactive by design, so what rules out interactive use is its speed rather than the interaction itself. Complete-model feasibility is established for the evaluated prompt, while encrypted task accuracy, service rate, and networked deployment remain open.

1 Introduction

Human genomic sequence carries an unusually persistent confidentiality risk. It can identify its contributor and, through familial relationships, reveal information about people who never supplied a sample; re-identification has been demonstrated even when conventional identifiers were removed [Gymrek et al. \[2013\]](#), [Erlich et al. \[2018\]](#). A disclosed genome also cannot be replaced in the manner of a password or access token. European data-protection law accordingly treats genetic data as a special category [European Parliament and Council of the European Union \[2016\]](#). The same sequence is the input from which genomic foundation models derive their value. DNAGPT is one released model of this class, serving classification, regression, and generation tasks through a common transformer backbone [Zhang et al. \[2023\]](#); the downstream tasks evaluated here are defined over human sequence. When a genomic model is available only as a service, conventional inference requires the data owner to send the sequence, or a representation of it, to the provider. Sending plaintext embeddings does not necessarily solve the privacy problem: recent work has shown that per-token representations from DNA foundation models can permit near-perfect reconstruction of the underlying sequence [Ouaari et al. \[2026\]](#). Homomorphic encryption offers a route for the provider to compute without receiving the genomic values in plaintext.

DNAGPT is public and can be run locally; we use it because its release makes the experiment reproducible and auditable. That is a requirement of the study rather than a convenience. An encrypted execution is only meaningful against the exact plaintext prediction it must reproduce, so the released weights are what make an independent numerical reference, and therefore an independent check of the encrypted result, possible at all. A model reachable only through an interface would permit no such reference. We therefore treat DNAGPT as a reproducible surrogate for a genomic model that a provider offers only as a service, whether for commercial, licensing, or governance reasons. Under that premise, the data owner’s alternatives are to disclose the genome, to decline the inference, or to use a protocol in which the provider computes without receiving the sequence.

We therefore ask whether a complete released genomic transformer can be executed under homomorphic encryption, and what limits its numerical fidelity, memory use, and runtime. We separate two questions. Feasibility asks

whether the complete computation can execute within the cryptographic and memory budget while meeting the predefined numerical criterion for the evaluated case. Practicality asks whether the resulting resource cost is suitable for the intended use. A numerically agreeing but slow execution can therefore establish complete-model feasibility for the tested case without establishing a practical service.

The main cryptographic difficulty arises from transformer operations that are not directly supported by low-depth CKKS arithmetic. CKKS efficiently supports the packed additions, multiplications, and rotations needed for dense linear transformations [Cheon et al. \[2017\]](#); LayerNorm, softmax, and GELU are not native CKKS operations. A non-interactive evaluator must approximate them with polynomial circuits, lengthening the multiplicative path and increasing the context, bootstrapping, and evaluation-key material that must be resident. Moving the exact functions to the key holder shortens those paths and resets the depth budget, but introduces interaction and a different threat model. Earlier systems establish both directions: THOR demonstrates non-interactive homomorphic transformer inference; THE-X and Sapphire use interactive workflows in which the client evaluates nonlinear functions [Moon et al. \[2025\]](#), [Chen et al. \[2022\]](#), [Biswas et al. \[2025\]](#). The division of labor is therefore established prior art. What has not been reported is an auditable account of the numerical behavior, memory, interaction, and executed work that results when it is applied end to end to a released genomic transformer, at that model’s own task prompt, with its weights and nonlinear formulas unmodified.

Figure 1 summarizes the resulting deployment, the division of work between the two parties, and the measured outcome.

We begin by establishing that the target computation is meaningful and independently reproducible. The released DNAGPT paths are evaluated across genomic-signal recognition, promoter and splice-site classification, and mRNA abundance regression, with task outputs frozen before encrypted work. The validation records the released float32 path, checks an independent NumPy float64 lineage against a float64-cast PyTorch shadow for every released block and the task head, and then compares encrypted execution with that reference. We build two CKKS designs: a non-interactive circuit with polynomial nonlinearities, and a client-assisted circuit whose fixed, non-adaptive boundaries evaluate the released nonlinearities exactly. Instrumented evaluations record where each design stops in correctness and memory, while runtime validation preserves the complete cryptographic operation schedule throughout the systems campaign.

The results separate three candidate barriers and resolve two of them. Numerical agreement holds: the client-assisted design evaluates all twelve released blocks and the task head at the 103-token prompt, and for the one input evaluated it reproduces the frozen plaintext label with 8.56×10^{-9} relative margin error against a 4×10^{-2}

A genomic fragment stays off the compute provider

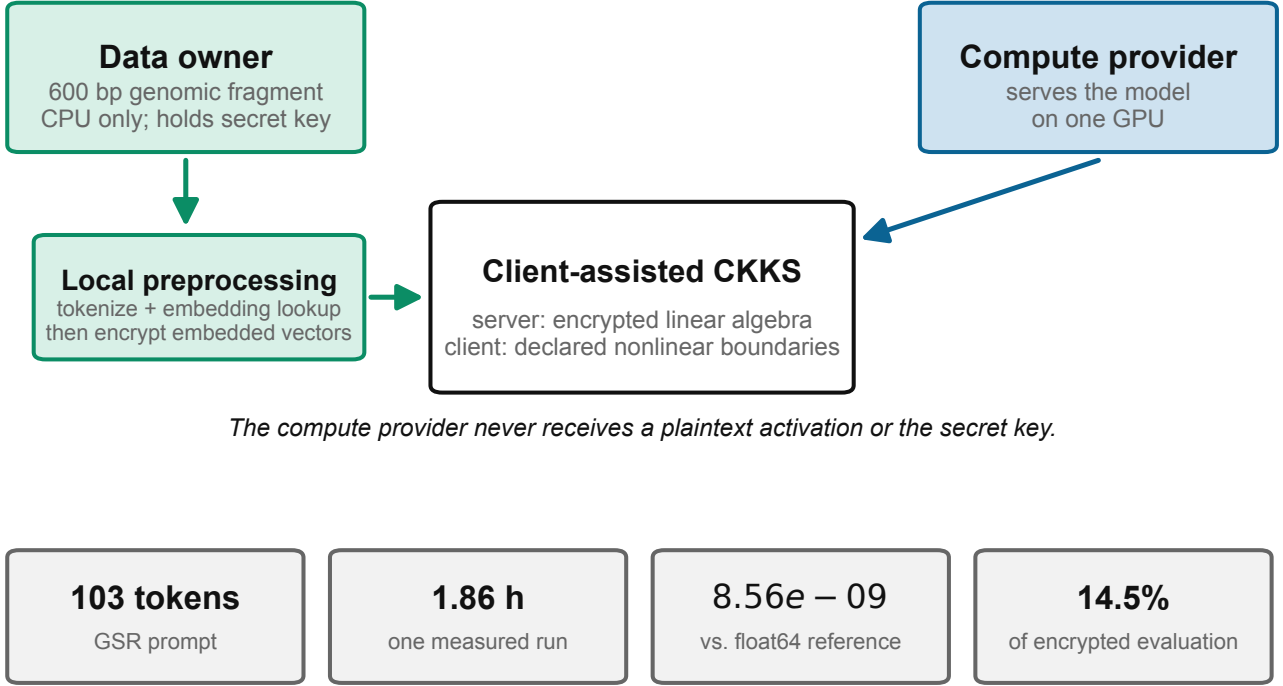


Figure 1: The data owner tokenizes and embeds a genomic fragment locally, encrypts the embedded vectors, and retains the secret key. The provider evaluates the encrypted model arithmetic between client boundaries on ciphertexts; the data owner evaluates the declared nonlinear functions on CPU. The measured implementation emulates both roles in one process. The result panel reports one measured execution at the 103-token genomic-signal prompt.

tolerance fixed before evaluation. Accelerator memory holds as well, and for a structural reason: each client boundary returns the computation to a freshly encrypted activation, so the depth the circuit must support is set by the longest stretch between two boundaries and not by the depth of the network, and device-wide GPU memory used does not grow across the twelve blocks. Bounded caching separately prevents encoded plaintexts from accumulating across host-side stages. The evaluated non-interactive design, by contrast, completes one released-weight block but exceeds the tested 80 GB envelope while loading its composition context and evaluation keys, placing that barrier in the nonlinear schedule rather than in the model’s dense maps. Cost is the barrier that remains: one encrypted classification takes 1.86 hours on an A100 GPU node, which does not enable interactive use in the evaluated form. These memory and timing figures are specific to the evaluated backend, packing, and parameters.

The paper makes the following contributions:

- **Complete-model graph feasibility.** All twelve released DNAGPT blocks and the task head execute at the 103-token task prompt from encrypted embedded vectors, with unmodified released weights and nonlinear formulas and without homomorphic bootstrap; the one evaluated prompt reproduces the ref-

erence label. To our knowledge this is the first complete forward pass of a released, pretrained genomic foundation model evaluated under CKKS; prior homomorphic transformer evaluations over biological sequence data have targeted small, purpose-trained classifiers [Harmanci et al. \[2024\]](#), [Chen et al. \[2024\]](#).

- **Independently checked numerical evaluation.** The protocol, packing, checked operation schedule, predeclared tolerance, and validation chain make the complete execution independently inspectable.
- **Measured systems anatomy and scoped negative results.** We report complete-model cost, interaction, depth, and memory; the stage-bounded cache that makes host-side reuse executable; and the memory boundary reached by the evaluated non-interactive configuration.

2 Background

2.1 DNAGPT and the genomic task families

DNAGPT is an autoregressive transformer adapted to sequence and numerical tasks over DNA [Zhang et al. \[2023\]](#). We evaluate its released 0.1-billion-parameter configuration. The backbone has 12 transformer blocks, hidden

width 768, and 12 causal-attention heads of width 64. Its feed-forward sublayer expands to width 3,072. DNA is represented through 6-mer tokenization and combined with task-format tokens before entering the transformer.

Each block follows the structure that determines the encrypted circuit: LayerNorm; query, key, and value projections; causal self-attention; an output projection and residual; a second LayerNorm; then an expanded MLP with GELU and another residual. The dense maps apply fixed model weights. LayerNorm and GELU apply nonlinear functions to the activations, while attention introduces products between two data-dependent quantities and normalizes them with a softmax.

The release includes fine-tuned heads for genomic-signal recognition and mRNA abundance regression, but none for the genome-understanding benchmark, so Section 4 evaluates that backbone with a locally fine-tuned linear head. All three task families predate DNAGPT and carry established task-specific baselines [Kalkatawi et al. \[2019\]](#), [Agarwal and Shendure \[2020\]](#), [Zhou et al. \[2024\]](#), against which Section 4 establishes model fidelity; the encrypted study then uses the signal-recognition path as its task-length anchor.

2.2 CKKS

CKKS supports approximate arithmetic on packed vectors of real or complex values [Cheon et al. \[2017\]](#). Encoding places many values into the slots of one plaintext, which is then encrypted under the data owner’s key. Addition and multiplication act slotwise across the packed vector, while rotations move values between slots. This single-instruction, multiple-data model is a natural match for transformer activations: a rotation-and-diagonal schedule can express a public matrix transform as ciphertext–plaintext multiplication and accumulation.

Not every transformer product has a public operand. Query–key scoring multiplies two encrypted projections of the input, and attention context multiplies encrypted attention weights by encrypted values. Those operations require ciphertext–ciphertext multiplication and relinearization. The distinction is about data ownership rather than implementation convenience: public weights and masks remain plaintext, while every query-derived operand remains encrypted at the server.

CKKS controls approximate fixed-point scale through rescaling. A multiplication consumes a level from a finite modulus chain, and the computation fails once that multiplicative budget is exhausted. Bootstrapping can refresh a ciphertext without decryption, while fresh client encryption can reset the budget in an interactive protocol. Both choices carry system costs, so multiplicative depth is an architectural resource rather than only a cryptographic parameter.

2.3 Why the nonlinearities are the architecture

CKKS directly supplies polynomial arithmetic, but a transformer requires inverse square root in LayerNorm, exponentiation and normalization in softmax, and GELU. A non-interactive circuit must replace these functions with polynomial approximations over declared input ranges. Higher-degree approximations consume additional multiplicative depth; across repeated blocks, the evaluator must provide a deeper context, a bootstrapping schedule, or both. Larger modulus chains and refresh operations also increase the context and evaluation-key material that must remain resident.

The alternative is to place exact functions at declared client boundaries, which removes approximation-range calibration and resets the encrypted depth budget but requires an online client and exposes intermediate activations to it. The choice between polynomial approximation and client evaluation therefore determines memory, interaction, depth, and the threat model. Section 5 evaluates both designs and reports where each stops.

2.4 Evaluated software backends

We separate correctness semantics from the performance path. OpenFHE provides CKKS context and security-parameter construction, encoding, rotations, polynomial evaluation, and approximate bootstrapping [Al Badawi et al. \[2022\]](#). It supplies the reference behavior used to validate local encrypted operators and the non-interactive design. The evaluated GPU backend interoperates with those representations and provides CUDA implementations of the server-side CKKS operations; all reported accelerator measurements use that path. Python, NumPy, and PyTorch construct and validate the frozen plaintext reference, but they do not participate in encrypted performance measurements.

We also evaluated the Boolean/integer-oriented TFHErs path and Concrete ML’s client/server workflow [Zama \[2022a,b\]](#); their available GPU acceleration and ciphertext expansion were less suitable for this matrix-heavy graph than the already validated CKKS path, and a separate CKKS wrapper lacked the refresh and general nonlinear-function support the non-interactive baseline needs. These decisions select among the software paths tested here. They do not establish that CKKS is the only scheme able to express encrypted transformer inference.

3 Deployment scenario and threat model

3.1 Deployment roles and privacy objective

The deployment has two parties but one cryptographic confidentiality objective. The data owner holds a human genomic sequence, ordinary CPU resources, and no GPU. It will not disclose the sequence to the compute provider. The model owner operates the accelerator infrastructure

and serves the DNAGPT model rather than distributing it to clients. Keeping the weights server-side is an operational preference of this deployment, not a model-confidentiality guarantee supplied by the protocol.

The released artifact does not itself require this arrangement; as Section 1 states, it serves as an auditable surrogate for a model whose owner limits access to a service.

The protected asset is the genomic input. The data owner encrypts its embedded numeric vectors and retains the secret key; the compute provider evaluates the served model on those ciphertexts. The evaluated input path assumes that the provider distributes the tokenizer and embedding table so that the data owner can construct those vectors locally, which is compatible with the public artifact used here but is an explicit service assumption: a provider unwilling to distribute those components would need a private token-index lookup, which is outside the evaluated protocol.

3.2 Local inference as a comparison

Local plaintext inference is preferable whenever the client may hold and run the model. It avoids all cryptographic work, and client assistance offers no computational advantage over it; in the complete encrypted inference of Section 7 the data owner alone spends 956.187 s evaluating LayerNorm inverse square roots, causal softmax, and GELU at the boundaries, and this study records no comparable local CPU forward-pass time. Where the client may not hold the weights, the choices are to disclose the genome, decline the inference, or use a privacy-preserving protocol. Client-assisted CKKS provides access to the served model without genomic disclosure; it is not intended to outperform plaintext inference.

The protocol assigns the encrypted model arithmetic between client boundaries to the party with accelerator hardware, while the key holder evaluates the exact nonlinearities on CPU, which protects the genomic input without requiring the data owner to operate a GPU. The allocation becomes more server-heavy as model width grows: at fixed sequence structure the client’s boundary work scales with the number of activation values, whereas the server’s dense transforms scale quadratically with hidden width, so the client fraction decreases as $O(1/D)$ with hidden width D . That is an asymptotic property of the work allocation, not a measurement of a wider DNAGPT model.

3.3 Adversary model

We consider a semi-honest compute provider under 128-bit classical CKKS parameters. It follows the prescribed circuit but may inspect everything it legitimately receives: public and evaluation keys, model parameters, ciphertexts, sequence length, message sizes, operation order, boundary schedule, and timing. It does not alter ciphertexts to probe the client or deviate from the declared boundary sequence.

Under this adversary, the compute provider receives no plaintext genomic vector, plaintext activation, partial decryption, or secret-key material. At a boundary it supplies a ciphertext and receives a fresh ciphertext under the data owner’s public key. The guarantee concerns the provider’s view of query-derived values; public model structure and plaintext weights remain available to the provider as inputs to ciphertext–plaintext operations.

Invariant 1 (Key custody). *The data owner generates and retains the secret key. No protocol message contains the key or a partial decryption, and the compute provider has no decryption operation.*

Invariant 2 (No plaintext at the server). *Every query-derived value held by the compute provider is a ciphertext under the data owner’s key. Client-evaluated nonlinearities return through fresh encryption rather than as decoded values.*

Invariant 3 (Non-adaptive boundaries). *The circuit and sequence length determine every boundary before execution; decrypted content does not select the next operation. The implementation validates the complete schedule and boundary counts for each accepted run and aborts on deviation. The boundary schedule is therefore fixed by public parameters rather than by the private input, though no separate leakage experiment compares transcripts across different inputs of the same length.*

3.4 What is not claimed

Model confidentiality is not claimed. The data owner observes full intermediate activations at every nonlinearity. This transcript creates a chosen-query extraction surface, and sufficiently diverse input–output pairs may make some affine segments identifiable. No extraction experiment or model-confidentiality proof is provided. Keeping weights on the server may deter casual copying, but it is not a cryptographic confidentiality guarantee. This limitation does not change the genomic-data objective, which protects the data owner from the compute provider.

Malicious servers, authenticated transport, production key custody, traffic analysis, timing and physical side channels, and compromised clients are outside the evaluated boundary. Encrypted scope begins after token-index lookup, so private lookup is also unresolved. Per-token embeddings remain sensitive: high-fidelity sequence reconstruction has been demonstrated for embeddings from other DNA foundation models Ouaari et al. [2026]. The embedded vectors are therefore encrypted here rather than sent in clear. The data owner sees its own intermediate activations and final output; privacy from the key holder is not an objective of this protocol.

4 Plaintext reference and model fidelity

An encrypted execution is useful only if the computation it reproduces is itself meaningful and independently

Table 1: Resource and trust allocation under local and served-model deployment. The plaintext service is the ordinary baseline that the encrypted protocol is intended to change.

	Client-local	Plaintext service	Client-assisted CKKS
Client must hold the full model	yes	no	no
Server receives genomic input	no	in plaintext	encrypted vectors
Client requires a GPU	deployment-dependent	no	no
Server evaluates model arithmetic	no	in plaintext	on ciphertexts
Client evaluates model nonlinearities	locally	no	at boundaries

checked. Task metrics answer the first question: they establish that the released model and locally trained heads retain the genomic capabilities for which the model is being evaluated. Per-example predictions answer the second. They are recorded before any encrypted work and become the frozen plaintext reference against which encrypted outputs are accepted.

The plaintext harness imports the upstream model implementation and preserves its classification and regression forward paths. It adds batching, metric computation, and result recording without changing model arithmetic. The promoter, splice-site, and abundance tasks use their documented evaluation splits. Genomic-signal recognition is evaluated on the full balanced source corpus, including examples used to train the released head, so its aggregate metric is release and pipeline fidelity evidence rather than held-out performance. Each run records the per-example outputs needed downstream.

The reference is not trusted merely because it is written in NumPy. The numerical chain has three explicit links. First, the released model’s default float32 path is recorded as a diagnostic lineage. Second, a deep-copied float64 PyTorch shadow is compared with an independent NumPy float64 re-derivation for every one of the 12 blocks and the classifier head; generation aborts unless relative and absolute differences satisfy 2×10^{-5} . Their largest per-block absolute difference is 5.22×10^{-6} . Third, encrypted execution is compared with the accepted NumPy float64 reference, as Section 7 reports. The released float32 path is not subject to the float64-shadow tolerance: its largest recorded absolute block drift from the NumPy lineage is 7.997×10^{-4} .

Table 2 shows that the target carries useful signal across classification and regression tasks. On the full polyadenylation-signal source corpus of 22,604 examples, the released classifier reaches 0.9124 accuracy and 0.916 F1, against the 0.9151 accuracy Zhang et al. [2023] report for this model class on a held-out quarter of that set. The locally fine-tuned understanding-benchmark heads reach MCC 0.680, 0.897, and 0.831 on core-promoter, promoter, and splice-site detection, against the 0.69, 0.87, and 0.85 that Zhou et al. [2024] report for DNABERT-2 itself on the same splits; other models in that benchmark score higher on two of the three. The released abundance-regression head reaches $r^2 = 0.562$ and Pearson $r = 0.753$, against the 0.62 Zhang et al. [2023] report over the Agarwal and Shendure [2020] regression baseline. These results establish model fidelity; encrypted evaluation is still required to establish cryptographic feasibility.

The genomic-signal aggregate is not presented as an independent generalization result because the full source corpus includes head-training examples.

Two dataset qualifications belong with these baseline results. DNAGPT does not release a head for the understanding benchmark, so we trained a linear head and evaluated the three human promoter and splice-site datasets selected for this study. For abundance regression, the historical source was recovered and its split procedure reproduced, but the resulting test set may not be gene-for-gene identical to the historical split. The abundance result is therefore model-fidelity evidence and is not used as an encrypted target.

4.1 Why 103 tokens

The encrypted anchor length follows from the genomic-signal task rather than from a convenient cryptographic setting. Its 600-base-pair window becomes 100 sequence tokens under the model’s 6-mer tokenization; three task-format special tokens bring the complete prompt to 103 tokens. The evaluated prompt is therefore the task input consumed by the released classifier, rather than a shortened surrogate chosen to fit the encrypted implementation.

The same arithmetic determines the other classification prompts. Core-promoter detection maps a 70-base-pair window to 12 sequence tokens plus one special token, or 13 total. The 300-base-pair promoter task uses 50 sequence tokens plus one special, or 51; the 400-base-pair splice task uses 67 sequence tokens plus one special, or 68. The 103-token genomic-signal prompt is the longest of these encrypted-scope tasks and consequently provides the anchor for the packing and sequence-length analysis in Sections 5 and 7.

5 Client-assisted CKKS protocol

5.1 Nonlinearity boundaries

CKKS carries the affine maps, reductions, residual additions, and attention products without exposing an activation to the compute provider. The difficulty lies in three nonlinearity classes: the inverse square root in LayerNorm, causal softmax, and GELU. We place a declared client boundary at each class. For LayerNorm, the server computes the encrypted mean and variance and sends only the variance term needed for the inverse square root.

Table 2: Released DNAGPT reproduced locally on three genomic task families, against published references. Signal recognition and abundance regression use the released fine-tuned heads; the understanding-benchmark rows use a locally fine-tuned head, as no head was released.

Task	Metric	This work	Reference	Source
Polyadenylation-signal recognition [‡]	accuracy	0.9124	0.9151	Zhang et al. [2023]
	F1	0.916	—	
Core promoter detection	MCC	0.680	0.69	Zhou et al. [2024]
300 bp promoter detection	MCC	0.897	0.87	Zhou et al. [2024]
Splice-site classification	MCC	0.831	0.85	Zhou et al. [2024]
mRNA abundance regression ^{†§}	r^2	0.562	0.62	Zhang et al. [2023]
	Pearson r	0.753	—	

[‡]The reference is DNAGPT’s own reported accuracy for this model class, measured on a held-out quarter of the set; this work evaluates all 22,604 examples. The comparison is same-model but not same-split.

[†]Model-fidelity evidence only. This task is outside the encrypted scope; see Section 9.

[§]The released regression head evaluated here is fine-tuned from the human-pretrained 0.1B variant, while the reference is reported for the mammal-pretrained variant, so the backbones differ.

Table 3: Notation. All values are those of the evaluated configuration.

Symbol	Value	Meaning
D	768	hidden width of the released model
H	12	attention heads, of $D/H = 64$ dimensions each
D_{mlp}	3072	MLP expansion width, $4D$
T	103	tokenized prompt length for the genomic-signal task
B	8	tokens packed per ciphertext (token lanes)
G	13	ciphertext groups, $\lceil T/B \rceil$
W	1024	feature width padded to a power of two
C	4	activation copies per ciphertext, D_{mlp}/D
N	65536	polynomial ring degree
$N/2$	32768	usable plaintext slots per ciphertext, $C \cdot W \cdot B$
L	13	multiplicative depth
λ	128	classical security level, in bits

For attention, it sends encrypted causal score tiles and receives encrypted softmax weights. The GELU boundary operates on the packed MLP activations.

At every boundary, the data owner decrypts values derived from its own query, evaluates the declared function exactly in double precision, and encrypts the result afresh. The data owner already holds the secret key and is the only party that decrypts. The compute provider receives neither a plaintext activation nor the secret key. Between these declared calls, the ciphertext lineage remains on the server and every model operation is evaluated homomorphically. Figure 2 shows the resulting evaluation order for one transformer block and which party performs each step.

5.2 Non-interactive design

The non-interactive design keeps a single ciphertext lineage and replaces every nonlinearity with a polynomial

approximation. It completed one released-weight transformer block, establishing that the block arithmetic can be expressed without intermediate decryption. The stopping point appeared when the same design was configured for multi-block composition: context and evaluation-key loading exceeded the tested 80 GB GPU memory envelope before useful composition arithmetic began.

The mechanism matters more than the failure location. Repeated polynomial approximation lengthens the multiplicative path, which requires a deeper modulus budget and a bootstrapping schedule, and the associated context and evaluation-key material must be resident before evaluation; composition therefore failed as a memory problem rather than as a numerical-correctness problem. Since published non-interactive systems evaluate encoders of comparable depth and width on a single accelerator at these parameters (Section 8), this reports where our non-interactive configuration stopped, not where the approach stops. It does identify the measured pressure that the client-assisted design removes: exact client evaluation shortens each encrypted segment and replaces encrypted bootstrapping with fresh encryption at already declared boundaries.

5.3 Protocol algorithms

Fresh encryption in Algorithm 2 returns the activation to level 0, so every client boundary fully resets the multiplicative budget. The configured depth of 13 therefore bounds the longest encrypted run between consecutive boundaries. It does not accumulate across the 12 transformer blocks.

The algorithms also expose the protocol transcript. Its order and size are determined by the circuit and sequence length, never by decrypted content. The evaluated block makes 857 physical client boundary crossings, including 91 score-tile decryptions and 727 weight-tile encryptions for attention. Attention therefore accounts for 818 of the 857 client calls, or 95.4%. In directional object accounting it contributes 818 of the block’s 896 boundary ciphertext objects, or 91.3%. These homogeneous counts identify attention as the main boundary-schedule target

Data owner

holds the genomic fragment and secret key · no GPU

Compute provider

holds the model · encrypted activations only · one A100

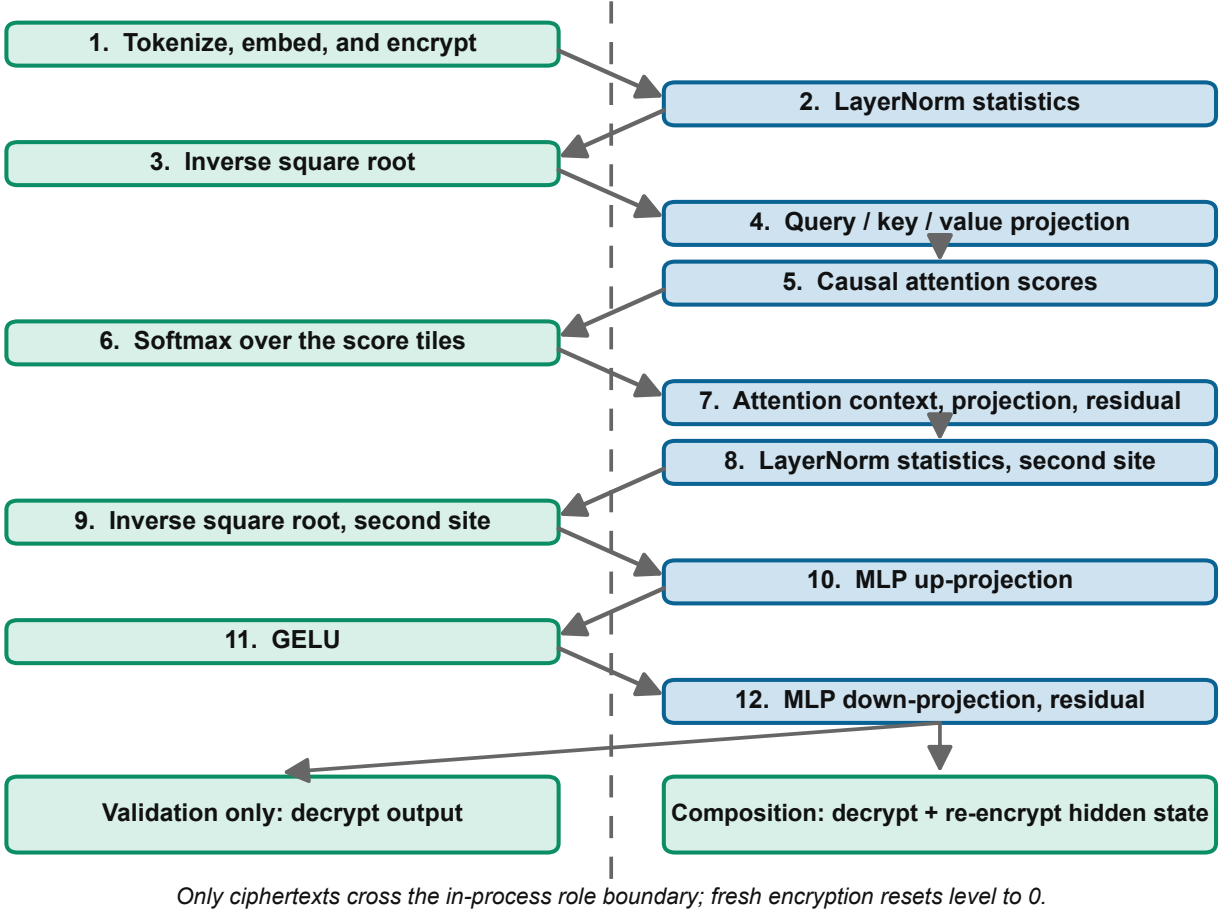


Figure 2: Evaluation order for one transformer block and party allocation. The provider computes both LayerNorm statistics under encryption; the data owner evaluates the inverse square root, causal softmax, and GELU at declared boundaries. A validation readout decrypts the output, while composition decrypts and freshly re-encrypts the hidden state before the next block.

without treating the implementation’s mixed-unit logical-instance counter as scalar work. The implementation validates the complete operation schedule, boundary counts, and remaining depth on every run and aborts when any differs. Section 6 can thus treat an unchanged circuit as a checked invariant rather than as an assurance.

5.4 Packing

Each ciphertext has 32,768 usable slots, indexed by activation copy, padded feature, and token lane. The layout assigns these slots as 4 copies by 1,024 features by 8 token lanes, with the token lane as the innermost index. Rotations by whole-lane strides can therefore move features without mixing tokens, while smaller rotations align tokens within a group. The released hidden width of 768 occupies part of the padded feature region; the remainder supplies the power-of-two width required by the linear transform and staging space for per-head attention scores. In dense transforms, rows and columns outside

the active width contribute zeros; padding therefore affects cost without changing the numerical result.

The four copies match the MLP expansion from width 768 to 3,072. They let its expansion chunks occupy parallel regions of one ciphertext rather than requiring a separate wide layout. The 103 tokens fill 13 ciphertext groups, with padding in the final group. Within a dense transform, a 32×32 baby-step giant-step decomposition covers the 1,024 padded features; baby rotations are shared where the schedule permits. Packing eight tokens together reduces the number of dense encrypted products from 1,236 serial-equivalent products to 156, a structural reduction of $7.92\times$. No latency factor follows from that operation count alone.

5.5 Causal attention

Attention follows a chunked lower-triangular schedule over the 13 token groups. Only causal group pairs are

Algorithm 1 Evaluating one transformer block. $G = 13$ token groups; $\odot$ is homomorphic multiplication; every operation outside a CLIENT call runs on ciphertexts.

Require: encrypted activations $c_0, \dots, c_{G-1}$; public weights W

Stage 1: normalization and projection

```

1: for  $g \leftarrow 0$  to  $G - 1$  do
2:    $n \leftarrow \text{LAYERNORM}(c_g, \gamma_1)$  ▷ Algorithm 3
3:    $\beta \leftarrow \text{BABYROTATIONS}(n)$  ▷  $N_1 - 1$  rotations, shared
4:    $Q_g, K_g, V_g \leftarrow \text{MATMUL}(\beta, W_q), \text{MATMUL}(\beta, W_k), \text{MATMUL}(\beta, W_v)$ 
5: end for
6: FLUSHENCODEDWEIGHTS ▷ bounds host memory to one stage

Stage 2: causal attention scores
7: for  $k \leftarrow 0$  to  $G - 1$  do
8:    $\tilde{K} \leftarrow \{\text{LANESHIFT}(K_k, \delta)\}$  for each  $\delta$  required by a later query group
9:   for  $q \leftarrow k$  to  $G - 1$  do
10:     $S \leftarrow 0$ 
11:    for  $\delta \in \text{ACTIVESHIFTS}(q, k)$  do
12:       $P \leftarrow Q_q \odot \tilde{K}_\delta$ 
13:      for  $h \leftarrow 0$  to  $H - 1$  do
14:         $S \leftarrow S + \text{REDUCESUM}(P \odot \text{mask}_h) \odot \text{isolate}(q, k, \delta, h) / \sqrt{d_h}$ 
15:      end for
16:    end for
17:    CLIENT.TAKESCORETILE( $S, q, k$ ) ▷ decrypt only
18:  end for
19: end for
20: CLIENT.SOFTMAX ▷ stable softmax over complete causal rows

Stage 3: attention context
21: for  $k \leftarrow 0$  to  $G - 1$  do
22:    $\tilde{V} \leftarrow \{\text{LANESHIFT}(V_k, \delta)\}$ 
23:   for  $q \leftarrow k$  to  $G - 1$ ;  $\delta \in \text{ACTIVESHIFTS}(q, k)$  do
24:      $A \leftarrow \text{CLIENT.EMITWEIGHTTILE}(q, k, \delta)$  ▷ re-encrypt only
25:      $C_q \leftarrow C_q + A \odot \tilde{V}_\delta$ 
26:   end for
27: end for

Stages 4 and 5: projection, MLP, residuals
28: for  $g \leftarrow 0$  to  $G - 1$  do
29:    $R_g \leftarrow c_g + \text{MATMUL}(\text{BABYROTATIONS}(C_g), W_{\text{proj}})$ 
30: end for
31: FLUSHENCODEDWEIGHTS
32: for  $g \leftarrow 0$  to  $G - 1$  do
33:    $\beta \leftarrow \text{BABYROTATIONS}(\text{LAYERNORM}(R_g, \gamma_2))$ 
34:    $h \leftarrow \sum_{c < 4} \text{MATMUL}(\beta, W_{\text{fc}}^c) \odot \text{copy}_c$ 
35:    $a \leftarrow \text{CLIENT.GELU}(h)$ 
36:    $c'_g \leftarrow R_g + \sum_{c < 4} \text{MATMUL}(\text{BABYROTATIONS}(a \odot \text{copy}_c), W_{\text{mlp}}^c)$ 
37: end for
38: return  $c'_0, \dots, c'_{G-1}$ 

```

Algorithm 2 The client boundary. The data owner is the only party holding sk , and every value it decrypts derives from its own query.

```

1: function CLIENTBOUNDARY( $c, f$ )
2:    $v \leftarrow \text{DECODE}(\text{DECRYPT}(\text{sk}, c))$ 
3:    $v' \leftarrow f(v)$  ▷ evaluated exactly, in double precision
4:   return  $\text{ENCRYPT}(\text{pk}, \text{ENCODE}(v'))$  ▷ fresh ciphertext at level 0
5: end function

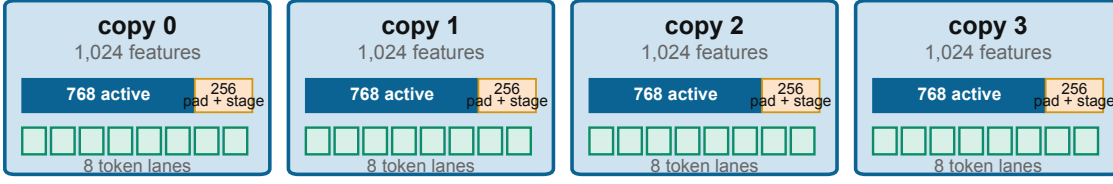
```

visited, producing 91 encrypted score tiles. Shifts needed by several later query groups are materialized per key group and reused. The client collects all score tiles before applying stable softmax over complete causal rows; normalization is therefore a global synchronization point in this design. It then emits the encrypted weight tiles consumed by the context computation.

Plaintext model weights do not eliminate ciphertext–ciphertext multiplication inside attention: both query–key scoring and softmax-weight–value aggregation multiply one activation-derived quantity by another. Of the block’s 1,506 ciphertext–ciphertext multiplications, LayerNorm contributes 26 centered-value squares and 26

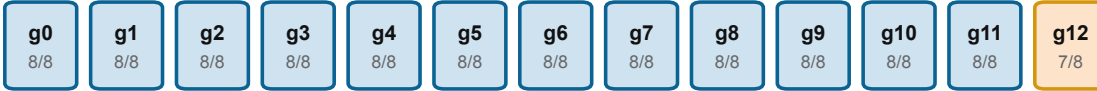
One ciphertext holds 32,768 slots

4 copies $\times$ 1,024 features $\times$ 8 token lanes = 32,768 slots. The copies carry the 4 $\times$ MLP expansion.



GSR: 600 bp become 103 tokens in 13 groups

600 bp $\div$ 6-mer = 100 tokens, plus 3 task specials = 103. 103 tokens across 8 lanes rounds up to 13 groups; the final group is partly padded.



The 13 groups give 91 lower-triangular causal score tiles, and just 1 of the 104 lanes is padding.

Figure 3: Slot layout for the genomic-signal task. Each ciphertext carries four copies of 1,024 features across eight token lanes: 768 features are active and the remainder supplies power-of-two padding and attention staging. The 103-token prompt occupies 13 groups and generates 91 lower-triangular causal score tiles.

Algorithm 3 LayerNorm. The server computes both statistics under encryption; only the inverse square root crosses the boundary. The released checkpoint carries no bias term.

```

1: function LAYERNORM( $x, \gamma$ )
2:    $\mu \leftarrow \text{REDUCESUM}(x) \odot (1/D)$ 
3:    $z \leftarrow x - \mu$ 
4:    $\sigma^2 \leftarrow \text{REDUCESUM}(z \odot z) \odot (1/D) + \varepsilon$ 
5:    $r \leftarrow \text{CLIENTBOUNDARY}(\sigma^2, t \mapsto t^{-1/2})$ 
6:   return  $(z \odot r) \odot \gamma$ 
7: end function
```

Algorithm 4 Packed matrix product, baby-step giant-step over the packed diagonals. $N_1 = N_2 = 32$ and $N_1 N_2 = W = 1024$.

```

1: function MATMUL( $\beta, M$ )
2:    $d \leftarrow \text{ENCODEDDIAGONALS}(M, \text{level}(\beta_0))$   $\triangleright$ 
   encoded once, cloned per use
3:    $y \leftarrow 0$ 
4:   for  $j \leftarrow 0$  to  $N_2 - 1$  do
5:      $u \leftarrow \sum_{i < N_1} \beta_i \odot \text{CLONE}(d_{N_1 j + i})$ 
6:      $y \leftarrow y + (j = 0 ? u : \text{ROT}(u, B N_1 j))$ 
7:   end for
8:   return  $y$ 
9: end function
```

products with client-returned inverse square roots. Attention contributes 727 query-key products and 727 weight-value products, accounting for 97% of the total. The latter term follows directly from returning exact attention weights as fresh ciphertexts; it is the arithmetic price paid to keep the attention nonlinearity exact at the client

boundary.

5.6 Depth and parameters

The retained CKKS context uses ring dimension 65,536, yielding 32,768 usable slots at a 128-bit classical security level. It uses 50-bit scaling, a 60-bit first modulus, 3 hybrid key-switching digits, and 80 rotation keys. Multiplicative depth 13 is the smallest demonstrated passing depth for this scale, digit count, ring, and packing. The block completes with 6 of the 13 levels remaining. The retained setting is the demonstrated feasible point in this parameter space; parameter optimization is a separate experiment.

5.7 Composition across blocks

Composition uses a declared full-hidden-state client refresh between blocks. The data owner decrypts the packed block output, restores its token-vector layout, and encrypts fresh level-0 inputs for the next block under the same context and key lineage. The complete evaluation uses that mechanism throughout, with 11 full-hidden-state refreshes and one last-token refresh into the task head; Section 7 reports the measured outcome.

6 Host-side systems work at fixed circuit

Encoding each distinct weight diagonal once removes redundant host work but creates a memory lifetime problem. An implementation that retained every encoded template was terminated by the operating system at

61,863 MiB resident, when the MLP began building templates on top of those cached by earlier stages. Bounding the cache to one live stage makes reuse executable within the tested host-memory envelope. Peak host memory is then determined by the largest stage, not by every encoded weight in the block. The remaining changes govern when host data are prepared, how long they remain resident, and where CPU work executes.

Every change in this section holds the encrypted circuit fixed: the retained implementation executes the same checked operation schedule before and after each change, with packing, multiplicative depth, boundary order, and the frozen plaintext reference unchanged, and the run-time checks of Section 5 abort on any deviation. No retained change removes a dense transform, reduces the number of attention tiles, moves linear algebra to the client, or relaxes the acceptance criterion. The section therefore reports verified changes in executed work and memory but no elapsed-time comparison, because no dedicated-node measurement of the pre-optimization configuration exists.

6.1 Host-side encoding redundancy

The baseline rebuilt and encoded every packed diagonal for every dense product. Its 156 products each required 1,024 diagonal plaintexts, producing 159,744 encoding operations per block, and about 90% of that work was byte-identical: the block has 12 weight matrices, each used across all 13 token groups. The GPU could not begin a multiplication until the host had prepared its plaintexts.

In the baby-step giant-step transform, a plaintext diagonal is not merely a view of the original matrix: it must be packed, scaled, and encoded for the ciphertext level at which multiplication will occur. Repeating that conversion for every token group made a model-invariant representation step dominate the call path. The optimization target is therefore the lifetime and placement of plaintext preparation, while the encrypted products and accumulations remain fixed.

6.2 Encode once, clone per use

The retained cache encodes each distinct weight diagonal into a pristine host template. A multiplication receives a lightweight clone that shares the encoded host data by reference while carrying fresh device state; the clone is loaded and evicted for that operation. This removes 90% of the 159,744 encoding operations without keeping a plaintext object resident on the GPU between multiplications.

Cloning is required by the evaluated backend’s object semantics. Its plaintext-load path returns early when an object is already marked as loaded, so directly reusing a template across multiplicative levels can pair a new ciphertext level with stale residue-number-system limbs. Fresh device state on each clone forces a load at the level required by the current operation. An isolated

GPU micro-test checked clone-versus-fresh-encode equality, cross-level reuse, stable device memory under repeated multiplication, and equality between parallel and serial encoding before the cache was applied to the complete block.

Templates are indexed by both weight matrix and multiplicative level, which preserves the backend’s level contract while still sharing work among token groups that reach the same matrix at the same level. The cache therefore stores reusable encoded data, not a ciphertext or a mutable GPU object, and leaves homomorphic multiplication itself unchanged.

6.3 Bounding host memory

Removing repeated encoding exchanges computation for host memory. An early cache retained templates from every completed stage, and the process did not survive the MLP. Two exact changes bound the cache. First, a template is encoded directly at its multiplicative level of use, so it carries only the limbs needed by the receiving ciphertext; this changes no arithmetic, because the backend would otherwise drop a level-0 plaintext to the same level before multiplication. Second, the implementation flushes the cache after query/key/value projection and after the attention projection, which are the final use sites for their respective weight sets: no weight matrix recurs across either boundary, so retaining those templates cannot produce another hit.

The lifetime policy follows the block topology. Within a stage, the group-major loop revisits its weights as it advances through token groups, so early eviction would discard real reuse; across stages, the next operation consumes a different set of weights, so eviction there releases memory without creating a later re-encoding.

The target-process resident-set trace follows this policy, as Figure 4(d) shows. Query/key/value projection reaches 25,530 MiB. Attention begins after a flush and uses 12,434 MiB. The MLP holds eight matrices and reaches the block peak of 48,610 MiB. The largest live stage, rather than every encoded weight in the block, determines peak host memory.

6.4 Placement and parallelism

Plaintext preparation also depends on where host work runs. The retained implementation binds threads to physical cores and uses NUMA-local allocation so encoding threads stay close to the memory they allocate and to buffers staged for the GPU; decryption, exact nonlinear evaluation, and re-encryption use the same host resources. Template construction is then batched across the available 32 cores with dynamic scheduling, since each diagonal can be encoded independently without changing the order or result of encrypted multiplications; parallel and serial construction were checked for equal outputs before the parallel path was retained. Affinity controls placement and batching exposes independent diagonals to the

cores, so together they make plaintext preparation a core-local parallel batch under the same cryptographic parameters. Their elapsed-time effect awaits a sourced paired measurement.

6.5 Remaining optimization targets

Mask plaintexts are still encoded afresh at approximately 17,400 uses per block. Head masks, score-isolation masks, lane masks, and copy-activity masks all have fixed values under the declared layout, so host templates could remove the same class of repeated work already eliminated for weight diagonals. This target remains unimplemented.

A second opportunity lies in the four packed activation copies. They currently carry duplicates through a dense transform. An analyzed layout would apply distinct transforms to those copies, placing query, key, and value projections together and likewise combining the MLP expansion chunks. At fixed model arithmetic, the projected schedule reduces dense products from 156 to approximately 52 and dense ciphertext–plaintext multiplications from 159,744 to approximately 53,248; the total ciphertext–plaintext count falls by roughly 60% before mask and copy-repair overhead. This is a derived projection for an unimplemented layout, and it requires an exact reconstruction proof before implementation.

Client work has a separate opportunity. Attention dominates the boundary schedule (Section 5), and each attention tile’s cryptographic work is independent. Concurrent client boundaries were unsafe under the evaluated shared crypto context, so the retained implementation executes them sequentially. A thread-safe context would expose this independent work to concurrency.

7 Results

The client-assisted circuit is evaluated along four axes: numerical agreement, executed work, memory, and complete-inference cost.

7.1 Evaluation setup and endpoints

The encrypted input is selected deterministically as the first record in the verified positive genomic-signal FASTA input. It is one 600-base-pair fragment, not a random, margin-stratified, or task-set sample. Local tokenization and embedding produce the 103-token prompt before encryption; the measured encrypted endpoint contains all 12 released blocks and the released task head.

The complete model is executed once on a whole-node allocation with one 40 GB A100 GPU and 32 CPU threads; the recorded telemetry shows no evidence of resource contention, although scheduler exclusivity was not requested. Timing starts before weight and reference loading and ends after final decryption. Server and client roles share one process and node, so boundary calls include cryptographic work but no serialization or network transport.

That single execution supplies every complete-model timing below; earlier contended timings are excluded.

7.2 Correctness

Encrypted output is accepted against the checked NumPy float64 reference of Section 4, never against the released float32 path, which is recorded separately. The separation distinguishes model-dtype drift from CKKS error and prevents an unchecked reimplement from judging its encrypted counterpart.

For one block at the 103-token task prompt, the decrypted output has relative infinity-norm error 4.64×10^{-9} against an acceptance tolerance of 4×10^{-2} fixed before the run, leaving seven orders of magnitude of headroom. The worst token-relative error is 4.7×10^{-9} , and the largest absolute deviation is 7.44×10^{-8} . These values establish numerical agreement for the encrypted block with the released weights and exact client nonlinearities.

The complete model is held to the same unchanged tolerance. Evaluating all 12 released blocks and the released task head at the 103-token task prompt yields a decrypted classification margin that differs from the manual NumPy float64 margin by 8.56×10^{-9} in relative terms and 1.05×10^{-7} in absolute terms. The encrypted, manual float64, and released float32 paths all produce label N for this prompt; relative to the released float32 margin, the encrypted margin differs by 1.61×10^{-6} .

Error accumulates with depth. The hidden-state relative infinity norm is 4.09×10^{-9} after the first block and 2.07×10^{-8} after the twelfth, while the largest absolute deviation grows from 6.56×10^{-8} to 7.97×10^{-6} , roughly two orders of magnitude across the model. Growth of that rate leaves the twelfth-block activation six orders of magnitude inside the acceptance tolerance. The classification-margin error is 6.67 orders below the same threshold. Approximation error accumulates, but it does not change the decision for the evaluated prompt.

7.3 Executed work

Every count in Table 5 is validated at run time under the checks of Section 5, so the schedule is evidence that the optimized implementation executes the same circuit rather than an assurance inferred from a passing output.

The counts locate the boundary cost in attention, as Section 5 sets out, and the benefit of token packing in the dense schedule: eight-token packing reduces the serial-equivalent dense-product count from 1,236 to 156, a $7.92\times$ reduction in operations. Rotations, masking, client work, and host preparation remain in the packed circuit, so that factor describes operation count rather than latency.

One block presents 130 ciphertext objects to the client and receives 766 in return. Across the complete model, ideal batching of independent calls within a protocol stage yields a lower bound of 65 sequential transport

Table 4: Host-side changes at fixed circuit. Every row leaves the operation schedule, packing, multiplicative depth, and frozen plaintext reference unchanged. Effects are verified changes in executed work or in memory; the table carries no elapsed-time column, because no dedicated-node measurement of the pre-optimization configuration exists.

Change	Verified effect	Status
Re-encode every plaintext per multiplication	159,744 encodings	baseline
Thread affinity, NUMA-local allocation	host placement changed	retained
Encode once, clone per use	removes 90% of encodings	retained
Parallel batched encoding	equal to serial encoding	retained
Encode at use level, flush between stages	peak 48,610 MiB RSS	enabling
Net	checked operation schedule unchanged	

Table 5: Checked operation schedule for the evaluated transformer block.

Operation	Count	Executor
Dense products	156	compute provider
Ciphertext–plaintext multiplications	177,734	compute provider
Ciphertext–ciphertext multiplications	1,506	compute provider
Rotations	8,173	compute provider
Slot reductions	8,776	compute provider
Physical client boundary crossings	857	data owner
Attention-related client calls	818 (95.4%)	data owner

phases including input and output. The present implementation instead performs 10,299 in-process boundary calls. Ciphertext levels, serialization representation, compression, and transport overhead were not recorded, so a byte count cannot be derived honestly from the object count.

7.4 Memory

Device-wide GPU memory used during the evaluating run reaches 9839 MiB, within the 40 GB device. The GPU was empty at allocation, making attribution to the run reasonable, but the sampler does not expose a per-process memory metric. Host memory follows the stage lifetime of encoded plaintext templates measured in Section 6, peaking in the MLP stage of each block rather than accumulating across the block.

The complete model confirms both bounds at full depth, as the resource trace in Figure 4(a) and (b) records. Device-wide GPU memory used rises during context and key loading, reaches 9839 MiB after roughly 400 s, and then holds that value exactly for the remainder of the run. This matches the single-block footprint, with no growth by block index. Host resident memory peaks at 50,930 MiB, consistent with the MLP-stage maximum recurring once per block instead of accumulating. Client assistance therefore removes the accelerator-memory barrier that stopped the evaluated non-interactive configuration, and the flat device profile across twelve blocks shows this follows from the depth bound rather than from the particular input.

7.5 Composition and evaluation boundary

Two consecutive released blocks compose at reduced sequence length through a full-hidden-state client refresh, reaching 8.48×10^{-11} relative error after the second block, with no growth in device-wide GPU memory used. The complete model uses that mechanism throughout: all 12 blocks and the task head run in one key lineage, separated by 11 full-hidden-state refreshes between consecutive blocks and one further refresh of the final token into the head. Multiplicative depth stays at 13 throughout, the same budget the single block requires, and the run performs no homomorphic bootstrap at any point. Depth is thus a property of the longest segment between two client boundaries rather than of network depth, which is what makes the twelve-block key lineage expressible at all under these parameters.

7.6 Cost of a complete encrypted inference

A complete encrypted inference takes 6683 s of wall clock, or 1.86 hours. Encrypted evaluation accounts for 6594.748 s of that, and setup is negligible at this scale: context construction and key generation together take under four seconds, and the single final decryption takes 0.044 s. The instrumented phases are a decomposition rather than a partition, leaving 59.504 s, or 0.9% of wall clock, outside them; Table 8 shows that remainder rather than distributing it silently.

Within encrypted evaluation the split between the two parties is exact (Figure 4(c)): the residual provider-side bucket uses 5638.561 s and the data owner’s boundary work uses 956.187 s, and the two sum to the evaluation total without residue. The provider bucket includes GPU-backend work and the host support around it; it is not GPU-kernel time. The compute provider therefore carries 85.5% of the encrypted-evaluation interval and the data owner 14.5%, with the data owner’s share running on CPU alone. Across 10,299 in-process boundary calls the mean call costs 0.093 s.

The twelve blocks account for 6565.993 s and the encrypted task head for 9.046 s, so the head is not a meaningful part of the cost. The twelve refreshes together cost 19.710 s, which places the price of the composition mechanism at roughly 0.3% of encrypted evaluation. Dividing block evaluation by twelve gives 547.166 s per block. The twelve individual block times span 533.869 s to 558.943 s,

Table 6: Derived ciphertext-object accounting for a complete inference. Counts follow from the checked protocol schedule; they are not observed network messages.

Direction or quantity	Count	Scope
Server to client	1,719	nonlinear and refresh boundaries
Client to server	9,339	nonlinear and refresh boundaries
Both directions	11,058	boundary ciphertext objects
Initial encrypted inputs	13	client to server
Final encrypted output	1	server to client
Total including endpoints	11,072	ciphertext objects
Minimum sequential phases	63	boundaries, with ideal same-stage batching
Minimum phases including endpoints	65	algorithmic lower bound
Serialized byte volume	unknown	no wire format was measured

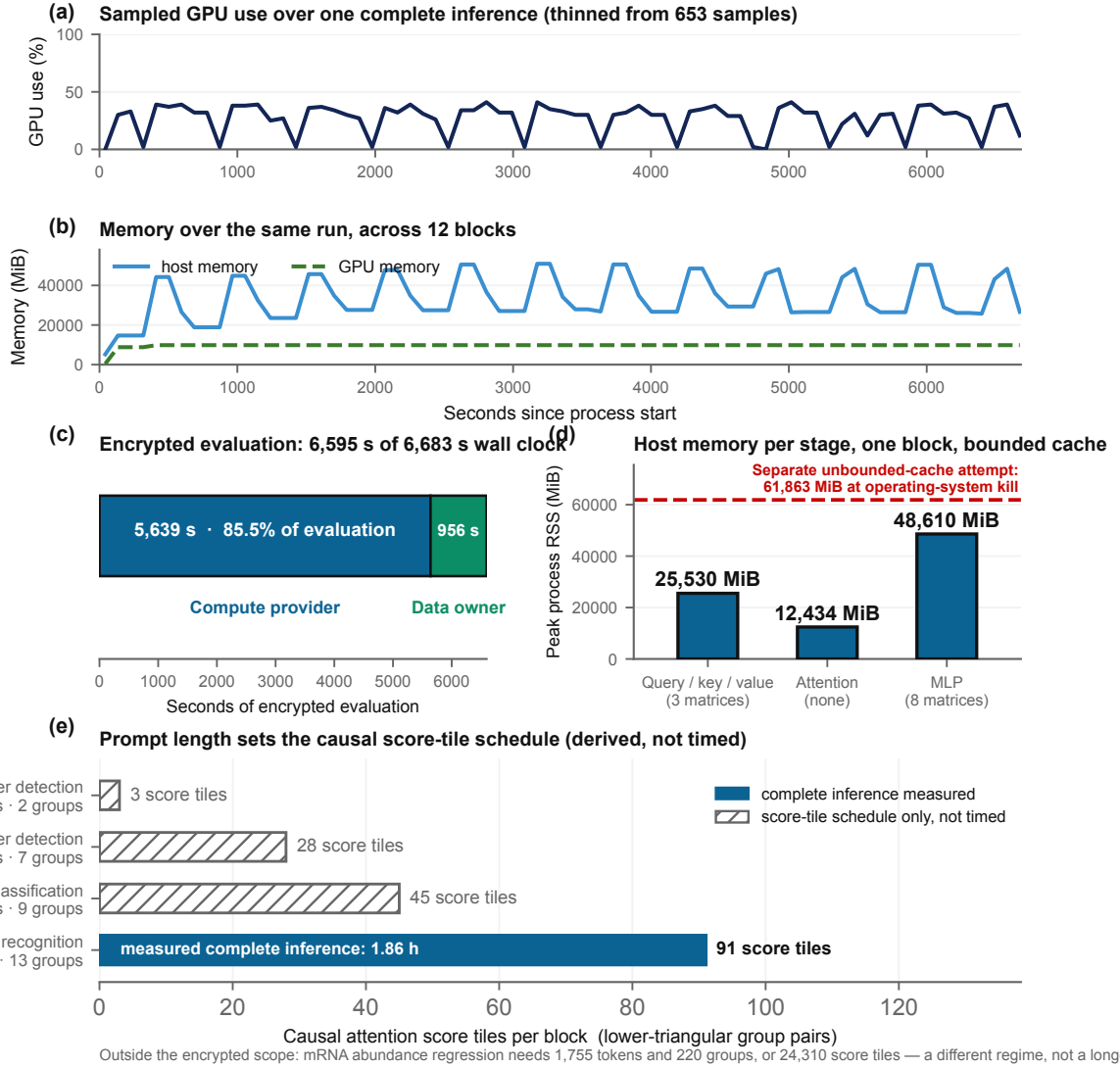


Figure 4: Systems behavior of one complete encrypted inference at the 103-token genomic-signal prompt. (a) Sampled GPU utilization reaches a maximum of 51%, which does not identify the limiting resource. (b) Host and device memory over the same run: host memory repeats the stage pattern of Section 6 once per block, while device-wide GPU memory used reaches its peak during the first block and does not grow with block index. Both curves are thinned from the full 653-sample trace, and the twelve cycles are the twelve blocks. (c) The encrypted-evaluation interval divided between the two parties. The provider-side bucket is the residual after client time and includes GPU-backend work plus host preparation, synchronization, and evaluator overhead; the data-owner bucket includes the declared nonlinear boundaries and full-hidden-state refreshes on CPU with no GPU. The two shares sum exactly to encrypted evaluation; setup and finalization remain in Table 8. (d) Stage host memory for one block under the bounded plaintext cache, with the separate unbounded-cache attempt that the operating system killed at 61,863 MiB. (e) Causal score-tile schedules across the encrypted-scope tasks, following from ciphertext groups rather than from a timing model; only genomic-signal recognition has a measured complete inference.

Table 7: Encrypted evaluation completed at each reported scope. Hidden-state error is a relative infinity norm over the activation; margin error is the scalar the classifier acts on. The two are different quantities and are kept in separate columns.

Scope	Status	Hidden-state error	Margin error
One block, 2 tokens	measured	—	—
Two consecutive blocks, 2 tokens, client refresh	measured	8.48×10^{-11}	—
One block, 103 tokens (task length)	measured	4.64×10^{-9}	—
Twelve blocks and task head, 103 tokens	measured	2.07×10^{-8}	8.56×10^{-9}
Encrypted token-index lookup	out of scope	—	—

Table 8: One complete encrypted inference: twelve blocks and the released task head at the 103-token task prompt, on one A100 GPU node using 32 CPU threads. Single execution on a whole-node allocation with no evidence of resource contention in the recorded telemetry; these are one sample’s values, not averages over repetitions. Shares are rounded independently and need not sum to 100%.

Phase	Seconds	Share of wall
Load released weights and verified reference	23.494	0.4%
Crypto context, key generation, GPU key upload	3.945	0.1%
Encrypt the embedded input	1.265	0.0%
Encrypted evaluation	6594.748	98.7%
provider-side encrypted evaluation	5638.561	84.4%
declared client boundaries and refreshes (CPU)	956.187	14.3%
Final decrypt	0.044	0.0%
Uninstrumented remainder	59.504	0.9%
Wall clock	6683	

a spread of 4.7% across architecturally identical blocks on one node. That bounds within-run scheduling jitter; it is not run-to-run variance, which a single execution cannot report.

7.7 Sequence-length structure

Prompt length follows from each task format, as Section 4 derives. Eight-token packing maps the 103-token anchor to 13 ciphertext groups; the shorter promoter and splice-site prompts use fewer groups under the same layout, yielding the smaller causal schedules shown in Figure 4(e).

The abundance-regression task lies outside the encrypted scope. Its 10,500 base pairs produce 1,755 tokens and 220 ciphertext groups under the same tokenization rule. The causal attention schedule grows quadratically in the number of groups, placing this task in a different circuit-size regime from the classification prompts.

7.8 Feasibility

Complete-model graph feasibility is established under the stated protocol: all 12 released blocks and the task head execute at the task’s 103-token prompt under one key

lineage with declared refreshes, at unchanged multiplicative depth and device memory and with no homomorphic bootstrap. These are measurements of the complete computation, not projections from a block.

Numerical generality has a narrower scope. The one selected prompt reproduces the frozen label with 8.56×10^{-9} relative margin error, but one prompt does not establish encrypted task accuracy or input-independent stability. The affirmative result is that the executed case meets the predeclared numerical criterion while composing the complete released graph.

Encrypted token-index lookup and network transport remain outside this verdict, as detailed in Section 9.

7.9 Practicality, judged separately

Feasibility does not make the current implementation practical. One encrypted classification of one 600-base-pair genomic fragment occupies an A100 node for 1.86 hours and requires an online key holder throughout, which rules out interactive per-query use in the evaluated form. Suitability for a specified non-interactive workload has not been measured.

Cross-paper results provide context rather than a controlled verdict. The 6683 s measured here is roughly an order of magnitude above the 602 s THOR reports for a non-interactive encrypted forward pass at comparable depth, width, and cryptographic parameters Moon et al. [2025], at a shorter prompt. Section 8 sets out the differences that make the comparison uncontrolled: this implementation is not performance-leading, and the gap does not isolate the cost of client assistance.

The measured trace does, however, indicate where that cost sits. Sampled GPU utilization (Figure 4(a)) has a median of 32% and a maximum of 51%; this does not identify the limiting resource, since the trace does not attribute wall time among encoding, synchronization, memory behavior, kernel efficiency, and boundary work. Read with the 85.5/14.5 provider–client split and the 0.3% cost of the refresh mechanism, this places the optimization target in the provider-side encrypted path rather than in the interaction the protocol adds. The practicality verdict is nonetheless negative for the evaluated form: it is too slow for interactive use, and throughput, serialization, and networked deployment remain unmeasured.

8 Related work

8.1 Private transformer inference

Private neural-network inference has long combined cryptographic tools according to the operation being evaluated. Gazelle established an influential hybrid design in which homomorphic encryption handles linear layers and garbled circuits handle nonlinearities [Juvekar et al. \[2018\]](#). Transformer systems inherited this decomposition but had to contend with attention, normalization, and activation functions at substantially greater scale. BOLT, Iron, Nimbus, and BumbleBee combine homomorphic encryption with secret sharing, oblivious transfer, and specialized two-party protocols for transformer inference [Pang et al. \[2024\]](#), [Hao et al. \[2022\]](#), [Li et al. \[2024\]](#), [Lu et al. \[2025\]](#). Their protocols optimize a joint secure computation between parties; the present work instead keeps the server’s linear path in CKKS and assigns exact nonlinear evaluation to the data owner at explicit boundaries. This choice changes both the security contract and the allocation of work between the parties.

The client-aided division of labor predates transformer work. nGraph-HE2 evaluates standard pre-trained networks under CKKS while returning to the client to compute their native activation functions, precisely so that those activations need not be replaced by polynomials [Boemer et al. \[2019\]](#), and THE-X carries the pattern to transformers: the server evaluates HE-compatible operations and the client decrypts and re-encrypts around nonlinear functions [Chen et al. \[2022\]](#). Safhire is the closest architectural antecedent [Biswas et al. \[2025\]](#): it likewise places homomorphically evaluated linear layers on the server and has the client decrypt an intermediate, apply the nonlinearity in plaintext, and return a fresh encryption, though it studies convolutional networks and adds randomized output permutation to make reconstruction from released intermediates harder. The client-assisted pattern is thus established prior art, and we adopt rather than propose it. What this study adds is its application to a complete forward pass of a released genomic transformer at that model’s task-defined prompt, preserving the released weights and exact nonlinearities, together with an independently checked plaintext reference that makes the resulting arithmetic, boundary schedule, and memory behavior reproducible.

The non-interactive line of work supplies useful cost context. THOR evaluates a 12-layer, 768-wide encoder at 128 tokens on a single A100, at ring degree 2^{16} , depth 13, and 128-bit security, in 602 s end to end, with no client interaction at any point [Moon et al. \[2025\]](#). NEXUS reaches non-interactivity over RNS-CKKS with SIMD ciphertext compression and a secure argmax [Zhang et al. \[2025\]](#); its headline per-inference figure is amortized over a batch and its single-prediction latency is substantially higher, so the two readings should not be conflated. MOAI reports a further 52.8% reduction in evaluation time against THOR by holding one packing convention across all layers and removing rotations from softmax and normal-

ization [Zhang et al. \[2026\]](#). Powerformer instead modifies the model, training HE-suitable polynomial components rather than preserving a released transformer unchanged [Park et al. \[2025\]](#), and ELLMo treats packing and depth as coupled design choices [Guzelhan et al. \[2026\]](#).

Model semantics, prompt length, nonlinear circuits, bootstrapping, packing, libraries, CPU paths, and measurement endpoints differ across these systems, so they establish that the present implementation is not performance-leading without isolating the cost of client assistance. The evaluated design occupies a different point in that space: it retains the released nonlinear formulas exactly, requires no homomorphic bootstrap, and pairs its execution with a frozen, independently checked plaintext reference.

8.2 Homomorphic encryption for genomics

Homomorphic encryption already has a substantial genomic-computing lineage. The iDASH secure genome analysis competition evaluated encrypted association studies, sequence search, and related privacy-preserving workflows [Kuo et al. \[2020\]](#). Private genomic-query systems combine homomorphic encryption with hashing and set-intersection techniques to test for variants without disclosing either the stored genome or the query [Çetin et al. \[2017\]](#). Those workloads center on searches or prescribed statistics over genomic records. DNAGPT inference instead composes dense projections, attention, normalization, and a feed-forward network while preserving the numerical behavior of a released foundation model. This study extends the genomic HE lineage to the feasibility boundary of that transformer forward pass.

The iDASH series has since moved from encrypted statistics toward encrypted neural evaluation, and supplies the closest prior art to this study. Its 2024 homomorphic-encryption track distributed DASHformer, a protein-family classifier comprising a tokenizer, an encoder, a single transformer block with four-head self-attention, and a dense classification layer, totaling 138,905 parameters over 50-residue fragments [Harmanci et al. \[2024\]](#). The winning solution evaluates it non-interactively under leveled CKKS, with no bootstrapping and no client participation at any point, classifying a batch of 163 encrypted sequences in about 165 s at 128-bit security [Chen et al. \[2024\]](#). On interaction that result is strictly stronger than the design evaluated here, which returns to the key holder at every nonlinearity. It reaches that strength under different constraints: one transformer block against twelve, a purpose-trained competition classifier against a released 0.1-billion-parameter backbone, and nonlinearities replaced by data-driven non-polynomial fits rather than preserved. To our knowledge, no released, pre-trained genomic foundation model has previously been evaluated under homomorphic encryption for a complete forward pass.

8.3 GPU acceleration and homomorphic matrix primitives

GPU CKKS libraries move number-theoretic transforms, key switching, rotations, and ciphertext arithmetic onto accelerators; the evaluated backend supplies those primitives and interoperates with the host cryptographic runtime Agulló-Domingo et al. [2025], Al Badawi et al. [2022]. Its API also shaped the available systems designs: the absence of ciphertext serialization prevented process-sharded evaluation, leaving thread-level concurrency and in-process device placement as the implementable paths. Recent multi-GPU transformer work treats placement and communication-computation overlap as first-class scheduling problems Gong et al. [2026], but the current implementation neither serializes ciphertexts between processes nor exercises a networked transport, so those paths must be evaluated directly before adopting a multi-device schedule.

Matrix layout determines which rotations, plaintext encodings, and reductions an encrypted linear layer requires. THOR’s diagonal-major representation and compact packing specialize that layout for transformer inference Moon et al. [2025]; HE-BLAS develops a broader library of homomorphic linear-algebra kernels and shape-aware algorithms Bae et al. [2026]. These works motivate the remaining targets in Section 6, as does MOAI’s rotation-free formulation of softmax and normalization Zhang et al. [2026]: reducing repeated packing work, copying fewer distinct diagonal plaintexts, and replacing segmented slot reductions with kernels matched to the tensor shapes. Because packing, key material, level consumption, and the operation schedule interact, each candidate primitive must be revalidated against the frozen plaintext reference and checked schedule before its performance is compared.

9 Limitations

The complete model has been evaluated once. All 12 released blocks and the task head run at the task’s full 103-token prompt on a whole-node allocation with no evidence of resource contention in the recorded telemetry. One sample supports a statement about the cost of that inference; it does not support a mean, a variance, or a service-rate claim, and the run has not been repeated. Nor does the study report encrypted task accuracy: the encrypted prediction is checked against the frozen plaintext reference for one input, not evaluated over a test set. That input was selected deterministically as the first record of the positive genomic-signal input, not randomly or by a predeclared margin stratum.

Interaction is a standing requirement rather than an implementation detail. The protocol needs an available key holder at every one of the 10,299 declared boundaries of a complete inference; a data owner that goes offline mid-evaluation has no way to finish it, and only a holder of the secret key can complete a boundary. A non-interactive evaluation does not require the key holder to be available

while the computation runs, and this study establishes no advantage over such designs in latency, throughput, or availability.

Client and server are roles inside one program on one node. Boundary calls perform the cryptographic operations directly, without ciphertext serialization, network transport, or a key custody service. A physical client boundary crossing is therefore not a network message. A derived schedule accounts for ciphertext objects and minimum dependency phases, but serialized byte volume, bandwidth sensitivity, transport latency, and production key handling remain unknown.

Encryption begins at embedded numeric vectors. Tokenization, token-index lookup in the embedding table, and construction of the initial embedding occur before the encrypted boundary. The evaluated deployment assumes that the tokenizer and embedding table are distributed to the client. Private lookup is required if a provider will not make those components available.

Model confidentiality remains operational rather than cryptographic. Because the data owner sees full activations at every nonlinearity, the transcript divides the network into shallow segments with known nonlinearities. This creates a chosen-query extraction surface, and some affine segments may be identifiable from sufficiently diverse queries. Attention exposes composite relationships, however, and this study provides neither an extraction experiment nor a general recovery claim. Server-side weight placement may discourage casual copying, but it is not a cryptographic model-protection mechanism.

CKKS decryption is approximate. The client never returns a decoded value to the server, only a freshly encrypted result, but the server selects the ciphertext supplied to each declared boundary. Security analyses of approximate homomorphic encryption show why adversarially selected decryption queries require care Li and Micciancio [2021]. The semi-honest threat model rules out deviations from the declared schedule, and the current re-encryption path applies no noise flooding. The measured single-block error of 4.64×10^{-9} leaves seven orders of magnitude inside the 4×10^{-2} acceptance tolerance, and the twelfth-block hidden state leaves six, but numerical headroom alone does not establish a secure flooding parameter. Selecting and validating that budget remains future work.

The adversary exclusions are defined in Section 3. The results do not extend beyond that threat model.

10 Conclusion

This study gives a split answer to the question of encrypted DNAGPT inference. Client-assisted CKKS executes all twelve released blocks and the task head at the 103-token prompt from encrypted embedded vectors. For the one input evaluated, the decrypted label matches the frozen reference and its margin has 8.56×10^{-9} relative error against the 4×10^{-2} predeclared tolerance. Graph fea-

sibility is established for the complete released computation; numerical generality beyond this input is not. Client refresh bounds depth by the longest encrypted segment, so device memory stays flat across the blocks and no homomorphic bootstrap is required, while a stage-bounded plaintext cache prevents host-memory accumulation.

Cost is the constraint that remains. One encrypted classification of one 600-base-pair sequence was measured at 1.86 hours on an A100 GPU node, which does not enable interactive use in the evaluated form. It is a single-process, single-input, single-execution result rather than a service-rate or network measurement, and sampled GPU utilization peaking at 51% shows spare accelerator capacity without identifying the limiting resource. The two verdicts are therefore separate, and both are informative: complete-model graph feasibility is established for the executed case, practicality is not, and the obstacle between them is engineering throughput on the provider’s encrypted path rather than an observed correctness failure.

Future work

Repeated execution comes first. The cost reported here is one sample, so a confidence interval for complete-model latency requires repeating the same inference on the same allocation; numerical generality requires additional prompts, and an attributable optimization gain requires a paired pre-optimization measurement on a dedicated node. The present run supports none of the three. Systems work could extend the encode-once strategy from weight diagonals to mask plaintexts. Distinct transforms across the packed activation copies are the larger circuit opportunity. Before mask and copy-repair overhead, the derived schedule for that layout projects a reduction of roughly 60% in the block’s total ciphertext–plaintext count. A thread-safe cryptographic context would remove one barrier to concurrent boundary processing. Protocol closure also requires a noise-flooding budget for re-encryption. A deployment study must add serialized network transport and private token-index lookup before measuring the complete service.

Ethics statement

This study involved no human participants, specimens, or private clinical records. It used public reference datasets with recorded provenance. The work makes no clinical-performance claim and does not present the prototype as a deployable privacy guarantee beyond its stated threat model.

Data availability statement

All evaluation datasets are public. Their original sources, versions, preprocessing, and any recovery route used are recorded in the repository’s data-provenance document.

Sequence data, model weights, and cryptographic keys are not redistributed.

Code availability statement

The evaluation harnesses, encrypted-evaluation implementation, environment definitions, and the evidence record for the paper are maintained at https://github.com/Georgakopoulos-Soares-lab/private_genomic_dnagpt, released under the MIT license. Upstream model code, released weights, and evaluation datasets retain the licenses of their original distributors and are not redistributed here.

References

- V. Agarwal and J. Shendure. Predicting mrna abundance directly from genomic sequence using deep convolutional neural networks. *Cell Reports*, 31(7):107663, 2020. doi: 10.1016/j.celrep.2020.107663.
- C. Agulló-Domingo, O. Vera-López, S. Guzelhan, L. Daksha, A. El Jerari, K. Shivdikar, R. Agrawal, D. Kaeli, A. Joshi, and J. L. Abellán. FIDESLib: A fully-fledged open-source fhe library for efficient ckks on gpus. In *2025 IEEE International Symposium on Performance Analysis of Systems and Software*, pages 365–367, 2025. doi: 10.1109/ISPASS64960.2025.00045. URL <https://arxiv.org/abs/2507.04775>.
- A. Al Badawi, J. Bates, F. Bergamaschi, D. B. Cousins, S. Erabelli, N. Genise, S. Halevi, H. Hunt, A. Kim, Y. Lee, Z. Liu, D. Micciancio, I. Quah, Y. Polyakov, R. V. Saraswathy, K. Rohloff, J. Saylor, D. Suponitsky, M. Triplett, V. Vaikuntanathan, and V. Zucca. OpenFHE: Open-source fully homomorphic encryption library. In *Proceedings of the 10th Workshop on Encrypted Computing & Applied Homomorphic Cryptography*, pages 53–63, 2022. doi: 10.1145/3560827.3563379.
- Y. Bae, J. H. Cheon, G. Hanrot, J. H. Park, and D. Stehlé. Fast homomorphic linear algebra with BLAS. *Journal of Cryptology*, 39:25, 2026. doi: 10.1007/s00145-026-09580-x. Preprint: arXiv:2503.16080.
- S. Biswas, P. Chartier, A. Dhasade, T. Jurien, D. Kerriou, A.-M. Kermarec, M. Lemou, F. Tranie, M. de Vos, and M. Vujašinovic. Practical and private hybrid ml inference with fully homomorphic encryption. *arXiv preprint arXiv:2509.01253*, 2025. URL <https://arxiv.org/abs/2509.01253>.
- F. Boemer, A. Costache, R. Cammarota, and C. Wierzynski. nGraph-HE2: A high-throughput framework for neural network inference on encrypted data. In *Proceedings of the 7th ACM Workshop on Encrypted Computing & Applied Homomorphic Cryptography*, pages 45–56, 2019. doi: 10.1145/3338469.3358944. URL <https://eprint.iacr.org/2019/947>.
- G. S. Çetin, H. Chen, K. Laine, K. Lauter, P. Rindal, and Y. Xia. Private queries on encrypted genomic data. *BMC Medical Genomics*, 10(Suppl 2):45, 2017. doi: 10.1186/s12920-017-0276-z.
- J. Chen, L. Yang, C. Yang, S. Wang, R. Li, W. Miao, W. Wu, L. Yang, K. Wu, and L. Dai. Secure transformer-based neural network inference for protein sequence classification. *Cryptology ePrint Archive*, Paper 2024/1851, 2024. URL <https://eprint.iacr.org/2024/1851>. First-place solution, iDASH 2024 Track 1.
- T. Chen, H. Bao, S. Huang, L. Dong, B. Jiao, D. Jiang, H. Zhou, J. Li, and F. Wei. THE-X: Privacy-preserving transformer inference with homomorphic encryption. In *Findings of the Association for Computational Linguistics: ACL 2022*, pages 3510–3520, 2022. doi: 10.18653/v1/2022.findings-acl.277. URL <https://aclanthology.org/2022.findings-acl.277/>.
- J. H. Cheon, A. Kim, M. Kim, and Y. Song. Homomorphic encryption for arithmetic of approximate numbers. In *Advances in Cryptology – ASIACRYPT 2017*, volume 10624 of

- Lecture Notes in Computer Science*, pages 409–437, 2017. doi: 10.1007/978-3-319-70694-8_15.
- Y. Erlich, T. Shor, I. Pe'er, and S. Carmi. Identity inference of genomic data using long-range familial searches. *Science*, 362 (6415):690–694, 2018. doi: 10.1126/science.aau4832.
- European Parliament and Council of the European Union. Regulation (EU) 2016/679 of the european parliament and of the council. Official Journal of the European Union, 2016. URL <https://eur-lex.europa.eu/eli/reg/2016/679/oj>.
- Z. Gong, R. Ran, F. Yao, and W. Wen. AEGIS: Scaling long-sequence homomorphic encrypted transformer inference via hybrid parallelism on multi-GPU systems, 2026. URL <https://arxiv.org/abs/2604.03425>. Accepted at ICS 2026.
- S. N. Guzelhan, L. Daksha, C. Agulló Domingo, G. Jonatan, J. Kim, J. L. Abellan, D. Kaeli, and A. Joshi. ELLMo: Packing- and depth-aware encrypted transformer inference. IACR Cryptology ePrint Archive, Report 2026/198, 2026. URL <https://eprint.iacr.org/2026/198>.
- M. Gymrek, A. L. McGuire, D. Golan, E. Halperin, and Y. Erlich. Identifying personal genomes by surname inference. *Science*, 339 (6117):321–324, 2013. doi: 10.1126/science.1229566.
- M. Hao, H. Li, H. Chen, P. Xing, G. Xu, and T. Zhang. Iron: Private inference on transformers. In *Advances in Neural Information Processing Systems*, volume 35, pages 15718–15731, 2022. doi: 10.52202/068431-1143.
- A. Harmanci, L. Chen, M. Kim, and X. Jiang. Descriptor: Benchmarking secure neural network evaluation methods for protein sequence classification (iDASH24). *IEEE Data Descriptions*, 1: 109–112, 2024. doi: 10.1109/ieeedata.2024.3482283.
- C. Juvekar, V. Vaikuntanathan, and A. Chandrakasan. GAZELLE: A low latency framework for secure neural network inference. In *27th USENIX Security Symposium (USENIX Security 18)*, pages 1651–1669, 2018. URL <https://www.usenix.org/conference/usenixsecurity18/presentation/juvekar>.
- M. Kalkatawi, A. Magana-Mora, B. Jankovic, and V. B. Bajic. Deepgsr: an optimized deep-learning structure for the recognition of genomic signals and regions. *Bioinformatics*, 35(7):1125–1132, 2019. doi: 10.1093/bioinformatics/bty752.
- T.-T. Kuo, X. Jiang, H. Tang, X. Wang, T. Bath, D. Bu, L. Wang, A. Harmanci, S. Zhang, D. Zhi, H. J. Sofia, and L. Ohno-Machado. iDASH secure genome analysis competition 2018: blockchain genomic data access logging, homomorphic encryption on GWAS, and dna segment searching. *BMC Medical Genomics*, 13(Suppl 7):98, 2020. doi: 10.1186/s12920-020-0715-0.
- B. Li and D. Micciancio. On the security of homomorphic encryption on approximate numbers. In *Advances in Cryptology – EUROCRYPT 2021*, pages 648–677, 2021. doi: 10.1007/978-3-030-77870-5_23. URL <https://eprint.iacr.org/2020/1533>.
- Z. Li, K. Yang, J. Tan, W.-j. Lu, H. Wu, X. Wang, Y. Yu, D. Zhao, Y. Zheng, M. Guo, and J. Leng. Nimbus: Secure and efficient two-party inference for transformers. In *Advances in Neural Information Processing Systems*, volume 37, pages 21572–21600, 2024. doi: 10.52202/079017-0680.
- W.-j. Lu, Z. Huang, Z. Gu, J. Li, J. Liu, C. Hong, K. Ren, T. Wei, and W. Chen. Bumblebee: Secure two-party inference framework for large transformers. In *Network and Distributed System Security Symposium*, 2025. doi: 10.14722/ndss.2025.230057.
- J. Moon, D. Yoo, X. Jiang, and M. Kim. THOR: Secure transformer inference with homomorphic encryption. In *Proceedings of the 2025 ACM SIGSAC Conference on Computer and Communications Security*, pages 3765–3779, 2025. doi: 10.1145/3719027.3765150. URL <https://eprint.iacr.org/2024/1881>.
- S. Ouara, J. Kreuer, and N. Pfeifer. How private are DNA embeddings? inverting foundation model representations of genomic sequences, 2026. URL <https://arxiv.org/abs/2603.06950>.
- Q. Pang, J. Zhu, H. Möllering, W. Zheng, and T. Schneider. BOLT: Privacy-preserving, accurate and efficient inference for transformers. In *2024 IEEE Symposium on Security and Privacy (SP)*, pages 4753–4771, 2024. doi: 10.1109/SP54263.2024.00130.
- D. Park, E. Lee, and J.-W. Lee. Powerformer: Efficient and high-accuracy privacy-preserving language model with homomorphic encryption. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 11090–11111, 2025. doi: 10.18653/v1/2025.acl-long.543. URL <https://aclanthology.org/2025.acl-long.543/>.
- Zama. Concrete ML: Privacy-preserving machine learning with fully homomorphic encryption. Software repository, 2022a. URL <https://github.com/zama-ai/concrete-ml>.
- Zama. TFHE-rs: A pure rust implementation of the TFHE scheme for boolean and integer arithmetic. Software repository, 2022b. URL <https://github.com/zama-ai/tfhe-rs>.
- D. Zhang, W. Zhang, Y. Zhao, J. Zhang, B. He, C. Qin, and J. Yao. Dnagpt: A generalized pre-trained tool for versatile dna sequence analysis tasks. *arXiv preprint arXiv:2307.05628*, 2023. URL <https://arxiv.org/abs/2307.05628>.
- J. Zhang, X. Yang, L. He, K. Chen, W.-j. Lu, Y. Wang, X. Hou, J. Liu, K. Ren, and X. Yang. Secure transformer inference made non-interactive. In *Network and Distributed System Security Symposium (NDSS)*, 2025. doi: 10.14722/ndss.2025.230868. Also IACR ePrint 2024/136.
- L. Zhang, X. Wang, J. J. Sim, Z. Huang, J. Zhong, H. Wang, P. Duan, and K.-Y. Lam. MOAI: Module-optimizing architecture for non-interactive secure transformer inference. In *The Fourteenth International Conference on Learning Representations*, 2026. URL <https://openreview.net/forum?id=qJn4HtTzhH>.
- Z. Zhou, Y. Ji, W. Li, P. Dutta, R. V. Davuluri, and H. Liu. Dnabert-2: Efficient foundation model and benchmark for multi-species genomes. In *The Twelfth International Conference on Learning Representations*, 2024. URL <https://openreview.net/forum?id=oMLQB4EZE1>.